

ObjectIR Language Specification

A portable, stack-based intermediate representation

Version 1.0
February 7, 2026

Contents

1	Introduction	2
2	Design Goals	2
3	Execution Model	2
3.1	Evaluation Stack	2
3.2	Arguments and Locals	2
4	Stack Discipline and Verification	2
4.1	Stack Balance Rules	2
4.2	Control Flow Merge Rules	2
4.3	Type Safety	3
4.4	Initialization Rules	3
4.5	Undefined Behavior	3
5	Instruction Set	3
5.1	Textual IR Format	3
5.1.1	Lexical Rules	3
5.1.2	Keywords and Declarations	3
5.1.3	Method Bodies	4
5.1.4	Primitive Types	4
5.1.5	Method References	4
5.2	Conventions	4
5.3	Opcode Summary	4
5.4	Opcode Aliases	6
5.5	Operands and JSON Shapes	6
5.6	Structured Conditions (JSON)	6
6	Examples	7
7	Notes and Edge Cases	8
8	Future Extensions	8
9	Revision History	8

1 Introduction

ObjectIR is a portable, stack-based intermediate representation designed for execution, analysis, and compilation across multiple runtimes. It emphasizes explicit control flow, verifiable semantics, and strong tooling support.

2 Design Goals

- Platform-agnostic execution
- Explicit and deterministic semantics
- Verifiable stack discipline
- Friendly to JIT, AOT, and interpreter backends

3 Execution Model

3.1 Evaluation Stack

ObjectIR uses an implicit evaluation stack. Most instructions consume zero or more values from the stack and may push results back onto it.

3.2 Arguments and Locals

Arguments and local variables are addressed by identifier. Instance methods implicitly receive a `this` argument.

4 Stack Discipline and Verification

This section defines mandatory rules that all valid ObjectIR programs must satisfy. These rules enable static verification and predictable execution.

4.1 Stack Balance Rules

- The evaluation stack is empty at method entry.
- At each `ret` instruction:
 - `void` methods must leave the stack empty.
 - Non-void methods must leave exactly one value matching the declared return type.
- No instruction may consume more stack values than are present.

4.2 Control Flow Merge Rules

At any control-flow merge point (e.g. end of `if`, loop headers):

- All incoming paths must have identical stack height.
- Stack value types must match exactly or be verifier-compatible.

4.3 Type Safety

- Stack operations are fully type-checked.
- Arithmetic instructions require compatible numeric types.
- Method calls must match the exact target signature.
- Field, local, and argument accesses must reference declared symbols.

4.4 Initialization Rules

- Local variables must be assigned before use.
- Fields must be initialized before being read.
- Constructors are responsible for establishing object invariants.

4.5 Undefined Behavior

Programs that violate verifier rules have undefined behavior. Backends may reject invalid IR or apply stricter validation.

5 Instruction Set

5.1 Textual IR Format

The C++ runtime includes a textual IR parser used by the tooling provided by the default ObjectIR stack. The format below reflects the concrete syntax consumed by `IRTextParser` and the JSON operands expected by the runtime. This is the required first option besides Fob format for saving ObjectIR to a file.

5.1.1 Lexical Rules

- Whitespace separates tokens; newlines terminate instruction lines.
- Line comments start with `//` and continue to the end of the line.
- Identifiers may include letters, digits, underscores, dots, and backticks (e.g. `List1`).
- String literals use double quotes and support escapes: `"`, `\n`, `\r`, `\t`.
- Numeric literals support integer and decimal forms; negative literals are written with a leading `-`.

5.1.2 Keywords and Declarations

- Keywords: `module`, `class`, `interface`, `struct`, `enum`, `method`, `constructor`, `field`, `property`, `static`, `virtual`, `abstract`, `private`, `public`, `protected`, `local`, `if`, `else`, `while`, `for`, `switch`, `case`, `return`, `implements`, `version`.
- Local declarations inside method bodies use `local name: type`.
- Labels are declared as `labelName:` and resolve branch targets.

5.1.3 Method Bodies

- A method body is a sequence of local declarations and instruction lines inside `{}`.
- Each instruction consumes the remainder of its line as operands; the parser stops at newline or a brace.
- Branch operands may be numeric instruction indices or label names; labels are mapped to instruction indices during parsing.

5.1.4 Primitive Types

- `void`, `bool`
- `int8`, `int16`, `int32`, `int64`
- `uint8`, `uint16`, `uint32`, `uint64`
- `float32`, `float64`
- `char`, `string`

5.1.5 Method References

Method references used by `call/callvirt` follow the textual shape:

```
1  TypeName.MethodName ( paramType1, paramType2 ) -> returnType
2
```

Constructors use `TypeName..ctor` as the method name in the text form.

5.2 Conventions

- Mnemonics are case-insensitive.
- Identifiers are case-sensitive.
- Stack effects are written as `inputs` `outputs`.

5.3 Opcode Summary

The following mnemonics are recognized by the Reference C++ runtime. Variants noted in the *Aliases* column map to the same opcode.

Mnemonic	Operands	Stack Effect	Notes / Aliases
Stack			
<code>nop</code>	—	$\emptyset \rightarrow \emptyset$	No operation.
<code>dup</code>	—	$a \rightarrow a, a$	Duplicates the top stack value.
<code>pop</code>	—	$a \rightarrow \emptyset$	Discards the top stack value.
Loads / Constants			
<code>ldarg</code>	<code>name</code>	$\emptyset \rightarrow v$	Loads argument by name.
<code>ldloc</code>	<code>name</code>	$\emptyset \rightarrow v$	Loads local by name.
<code>ldfld</code>	<code>Type.Field</code>	$obj \rightarrow v$	Uses instance on stack; falls back to this if available.
<code>ldstr</code>	<code>"text"</code>	$\emptyset \rightarrow string$	String literal.
<code>ldc</code>	<code>literal</code>	$\emptyset \rightarrow number$	Integer constant (text form).
<code>ldi4</code>	<code>literal</code>	$\emptyset \rightarrow int32$	Alias of integer constant load.

Mnemonic	Operands	Stack Effect	Notes / Aliases
ldi8	literal	$\emptyset \rightarrow int64$	Alias of 64-bit integer constant load.
ldr4	literal	$\emptyset \rightarrow float$	Alias of 32-bit float constant load.
ldr8	literal	$\emptyset \rightarrow double$	Alias of 64-bit float constant load.
ldc.r4	literal	$\emptyset \rightarrow float$	Float constant.
ldc.r8	literal	$\emptyset \rightarrow double$	Double constant.
ldtrue	–	$\emptyset \rightarrow true$	Push boolean true.
ldfalse	–	$\emptyset \rightarrow false$	Push boolean false.
ldnull	–	$\emptyset \rightarrow null$	Push null.
Stores			
starg	name	$v \rightarrow \emptyset$	Stores to argument by name.
stloc	name	$v \rightarrow \emptyset$	Stores to local by name.
stfld	Type.Field	$obj, v \rightarrow \emptyset$	Pops value then instance; falls back to this when needed.
Arithmetic / Unary			
add	–	$a, b \rightarrow (a + b)$	String concatenation when either operand is string. Division by zero throws. Integer-only.
sub	–	$a, b \rightarrow (a - b)$	
mul	–	$a, b \rightarrow (a \times b)$	
div	–	$a, b \rightarrow (a / b)$	
rem	–	$a, b \rightarrow (a \bmod b)$	
neg	–	$a \rightarrow (-a)$	
Comparisons			
ceq	–	$a, b \rightarrow (a = b)$	
cne	–	$a, b \rightarrow (a \neq b)$	
clt	–	$a, b \rightarrow (a < b)$	
cle	–	$a, b \rightarrow (a \leq b)$	
cgt	–	$a, b \rightarrow (a > b)$	
cge	–	$a, b \rightarrow (a \geq b)$	
Control Flow			
ret	–	$[v] \rightarrow \emptyset$	Returns top of stack (or void if empty).
br	label index	$\emptyset \rightarrow \emptyset$	Unconditional branch.
brtrue	label index	$c \rightarrow \emptyset$	Branch if condition is truthy.
brfalse	label index	$c \rightarrow \emptyset$	Branch if condition is falsey.
beq	label index	$a, b \rightarrow \emptyset$	Branch if $a = b$.
bne	label index	$a, b \rightarrow \emptyset$	Branch if $a \neq b$.
bgt	label index	$a, b \rightarrow \emptyset$	Branch if $a > b$.
blt	label index	$a, b \rightarrow \emptyset$	Branch if $a < b$.
bge	label index	$a, b \rightarrow \emptyset$	Branch if $a \geq b$.
ble	label index	$a, b \rightarrow \emptyset$	Branch if $a \leq b$.
Object / Call			
newobj	Type	$\emptyset \rightarrow obj$	Allocates object instance.
call	method ref	$[args] \rightarrow [ret]$	Static call.
callvirt	method ref	$obj, [args] \rightarrow [ret]$	Virtual call; consumes instance.
castclass	Type	$obj \rightarrow obj$	Throws on invalid cast.
isinst	Type	$obj \rightarrow obj null$	Returns null if not instance.
Arrays			
newarr	Type	$len \rightarrow arr$	Allocates array with length from stack.
ldlen	–	$arr \rightarrow len$	Reads array length.
ldelem	–	$arr, idx \rightarrow elem$	Loads array element.
stelem	–	$arr, idx, val \rightarrow \emptyset$	Stores array element.

Mnemonic	Operands	Stack Effect	Notes / Aliases
Structured Control (JSON + IR Text)			
if	blocks	$cond \rightarrow \emptyset$	Operand provides then/else blocks.
while	condition+body	$\emptyset \rightarrow \emptyset$	Operand provides condition and body.
break	–	$\emptyset \rightarrow \emptyset$	Breaks out of structured loop.
continue	–	$\emptyset \rightarrow \emptyset$	Continues structured loop.
throw	–	$\emptyset \rightarrow \emptyset$	Reserved; currently not implemented.

5.4 Opcode Aliases

The runtime accepts the following alias mnemonics when decoding JSON or text IR:

- `ldcon` and `ldc` are equivalent.
- `ldi4`/`ldi32`/`ldc.i4` map to integer constant load.
- `ldi8`/`ldi64`/`ldc.i8` map to 64-bit integer constant load.
- `ldr4`/`ldc.r4` map to 32-bit float constant load.
- `ldr8`/`ldc.r8` map to 64-bit float constant load.
- `beq.s`, `bne.s`, `bne.un`, `bgt.s`, `bgt.un`, `blt.s`, `blt.un`, `bge.s`, `bge.un`, `ble.s`, `ble.un` are treated as their base comparison branches.

5.5 Operands and JSON Shapes

For JSON IR, instruction operands are encoded as objects. The following shapes are required by the C++ runtime:

- `ldarg/starg`: { `argumentName`: "name" }
- `ldloc/stloc`: { `localName`: "name" }
- `ldstr/ldc`: { `value`: <literal>, `type`: "string|int32|float64|..." }
- `ldfld/stfld`: { `field`: "Type.Field" } or { `field`: { `declaringType`, `name`, `type` } }
- `call/callvirt`: { `method`: { `declaringType`, `name`, `parameterTypes`: [...], `returnType` } }
- `newobj/newarr/castclass/isinst`: { `type`: "Type" }
- `Branches`: { `target`: "label" } or { `target`: 12 } or { `offset`: 12 }
- `if`: { `thenBlock`: [instructions], `elseBlock`: [instructions] }
- `while`: { `condition`: <Condition>, `body`: [instructions] }

5.6 Structured Conditions (JSON)

Conditions are encoded as objects with a `kind`:

- { `kind`: "stack" }: consumes a boolean-like value from the stack.
- { `kind`: "binary", `operation`: "ceq|cne|clt|cle|cgt|cge" }: compares two stack values.
- { `kind`: "expression", `expression`: <instruction> }: evaluates a single instruction to a boolean-like value.

6 Examples

simple hello world

```
1 module HelloWorld version 1.0.0
2 class Program {
3     ldstr "hello world"
4     call System.Console.WriteLine(string) -> void
5     ret
6 }
7
```

ObjectIR's hello world might seem a bit daunting at first if you have ever worked with CSharp decompilation.

notice how there isn't any "[square brackets?]".

that's because ObjectIR provides a more "python" like experience when it comes to it's importing style.

```
1 module HelloWorld version 1.0.0
2 class Program {
3     ldstr "hello world"
4     call System.Console.WriteLine(string) -> void
5     ret
6 }
7
```

```
1 module HelloWorld version 1.0.0
2 class Program {
3     ldstr "hello world"
4     call System.Console.WriteLine(string) -> void
5     ret
6 }
7
```

```
1 module HelloWorld version 1.0.0
2 class Program {
3     ldstr "hello world"
4     call System.Console.WriteLine(string) -> void
5     ret
6 }
7
```

```
1 module HelloWorld version 1.0.0
2 class Program {
3     ldstr "hello world"
4     call System.Console.WriteLine(string) -> void
5     ret
6 }
7
```

```
1 module HelloWorld version 1.0.0
2 class Program {
3     ldstr "hello world"
4     call System.Console.WriteLine(string) -> void
5     ret
6 }
7
```



```
1  module HelloWorld version 1.0.0
2  class Program {
3      ldstr "hello world"
4      call System.Console.WriteLine(string) -> void
5      ret
6  }
```

```
1  module HelloWorld version 1.0.0
2  class Program {
3      ldstr "hello world"
4      call System.Console.WriteLine(string) -> void
5      ret
6  }
```

7 Notes and Edge Cases

This section records undefined behavior, verifier caveats, and backend-specific constraints.

Json Support for ObjectIR Runtimes are to be removed in a later date as Json is much too inefficient for storing programs, the switch to Fob is recommended if you want small file size or Text IR for everything else.

8 Future Extensions

9 Revision History

Version	Date	Notes
1.0	January 18, 2026	Initial ObjectIR language specification